

Three ways to run an AI team: the five-task benchmark, rerun after the fixes

Run on 28 September 2026. The first benchmark is docs/eval/bench5/REPORT.md; its tasks, rubrics and scoring rules are unchanged here.

1. Summary

What we did. The first benchmark showed gaps in all three team designs. We fixed nine of them, plus one found during this rerun. Then we reran the same five tasks: 3 runs per task per architecture, 45 runs in all. Same model (Gemma 4, 31B), same tools, same limits. Box 2 drafted each team once with the fixed prompts, and all three architectures used that same team, plan and tool picks.

The fixes, one line each.

#	Fix
D63	Action Observer: after each wave, plain code looks for trouble; if there is any, one planner call may change the steps that have not run yet, and plain code decides whether the change is allowed.
D64	Shortened outputs keep the head, the tail and every result line, so a final count is never cut.
D65	Check steps get tools to re-check (run code, open files, re-fetch pages) and see the raw results of the steps they check.
D66	A number that came from the calculator or a program is no longer tagged "unverified".
D67	"Today / current / latest" tasks ask for the newest dated figure; plain code flags a figure older than 3 days.
D68	The planner sees every tool the team will really have; a file format is a skill, not a tool; things only the user can supply become open questions.
D69	A request that names a file format (xlsx, docx, pptx, pdf) gets the vetted local skill for it first.
D70	Harness fixes for the two baselines: AutoAgents' file blocks become real file writes; reply readers no longer cut answers or accept a bare tool request as an answer; one tool pick per task, shared.
D71	LibreOffice works headless in the sandbox, so spreadsheet formulas can be recalculated.
D72	<i>(found in this rerun)</i> A ## heading inside a role's prompt no longer splits the planner's reply; the loan task's first draft lost all its roles to it.

Results (15 runs per architecture; "solved" means right answer, required file right, no honesty problem that changes the answer):

Task	AutoAgents	AgentVerse	Amoeba
1. Loan payment	3/3 solved	3/3 solved	3/3 solved
2. Prime-number program	0/3 (3 partly)	1/3 (2 partly)	0/3 (3 partly)
3. Shipment spreadsheet	3/3 solved	1/3 (2 partly)	3/3 solved
4. Company CEO, with a source	3/3 solved	3/3 solved	3/3 solved
5. Diesel price and fuel sheet	0/3 (3 partly)	0/3 (3 failed)	3/3 solved

	AutoAgents	AgentVerse	Amoeba
Runs solved / partly / failed	9 / 6 / 0	8 / 4 / 3	12 / 3 / 0
Honesty problems	3	9	0
Files made correctly (6 runs asked for one)	4 of 6	3 of 6	6 of 6
Tokens, all 15 runs	507 thousand	1.59 million	1.04 million
Model calls	125	310	204
Time, sum of the 15 runs	92 min	228 min	191 min

Cost: about **\$3.68** at the high price estimate: \$3.52 for the 45 runs, \$0.16 for the drafts and tool picks (including the loan draft that failed before D72). Under the \$10 limit. No run crashed, so none was rerun.

Side by side with the first benchmark (one run per task then, three now):

	First benchmark (5 runs each)	This rerun (15 runs each)
AutoAgents solved	2 of 5 (40%)	9 of 15 (60%)
AgentVerse solved	2 of 5 (40%)	8 of 15 (53%)
Amoeba solved	3 of 5 (60%)	12 of 15 (80%)
Honesty problems per run (AA / AV / Am)	0 / 0.4 / 0.2	0.2 / 0.6 / 0
Model calls per run (AA / AV / Am)	8.4 / 18 / 15.6	8.3 / 20.7 / 13.6
Minutes per run (AA / AV / Am)	8.6 / 12.8 / 18.4	6.1 / 15.2 / 12.7

What came from the fixes, and what may be noise.

- *From the fixes (the mechanism is visible in the traces):*
 - AutoAgents now makes the spreadsheet: 3/3, against "no file" before. Its file blocks became real writes (D70) in all six file runs.
 - AutoAgents' diesel answers are no longer cut off (D70): failed before, partly now.
 - Amoeba's diesel went from "week-old price" to 3/3 solved. Two fixes did it: the research asked for the newest dated figure (D67), and in one run the check step caught a wrong price that the Action Observer then routed around (D63, D65).
 - Amoeba's primes count now reaches the answer instead of being cut (D64).
 - The loan answer is no longer covered in "unverified" tags (D66).
- *Probably noise, or not from our fixes:*
 - AgentVerse's one solved primes run.
 - AgentVerse's diesel is still 0/3: its single writer never produced a file. The failure has the same shape as before.
 - The honesty count rose for both baselines. With three runs per task we simply see more of their habit of describing work that did not happen: files, recalculations, checks.

2. The tasks

Task 1: the loan payment (\$250,000, 7% APR, 5 years; expected \$4,950.30 and the formula)

Plan: the same three roles and order as the first draft: analyst → checker → writer. This time both the analyst and the checker asked for a Python runner, because the planner now saw it was available (D68).

Runs: all nine solved.

- AutoAgents was cheapest (3–5 calls). One run did no tool call at all and got \$4,950.31 by its own arithmetic, within tolerance.
- AgentVerse used 14–15 calls per run. Its written working still had a rounding slip, but the stated payment was right.
- In Amoeba the analyst ran the formula in Python and the check step re-ran it. The answer tags the payment with the program's result ([S1]) instead of "unverified".

Action Observer: one call in each Amoeba run. The trigger was "QA Reviewer now holds local:Bash, which the plan never named". That tool was granted by our own D65 fix. The decision was CONTINUE, accepted, and nothing changed. The trigger was spurious (see section 4).

Task 2: the prime-number program (expected 1,229 and 9,973, from a real run, with the program shown)

Plan: same shape as before: engineer → checker → writer. The checker now also asked for web search.

Runs: only one of nine was solved; nearly every run failed the same way.

- AutoAgents, 3 partly:
 - In two runs the engineer sent AutoAgents' >>>file block to the shell, where it fails. Our D70 adapter only catches such a block when it is sent as a file write. So nothing ran, and the checker took the numbers from the web.
 - The third run really ran the program. None of the three answers shows the program.
- AgentVerse: 1 solved, 2 partly. All three runs really ran the program. In the solved run, four review rounds improved the sieve, and the writer ran each version and showed the last one.
- Amoeba, 3 partly:
 - The program ran and the check step wrote and ran its own checker (D65).
 - The count now reaches the answer (D64).
 - But the writer, which is told to add nothing new, named prime_finder.py instead of pasting the program.

Action Observer: no trigger, no call.

Task 3: the shipment spreadsheet (expected an .xlsx with totals 43 loads, \$79,800, and 1,855.81 average by formula)

Plan: smaller than before: three steps instead of four, with a data engineer, a checker and a writer. The format is now requested as the **xlsx skill**, not an invented "excel_generator" tool (D68). The skill was attached in every run (D69).

Runs: all nine files were correct when opened and recalculated. The answers differed.

- AutoAgents: 3 solved.
 - The file blocks became real writes (D70).

- In one run the checker "verified" by imagining its own tool result, which is 1 honesty problem.
- AgentVerse: 1 solved, 2 partly, 5 honesty problems.
 - Its reviewers asked for "robustness" features: an Excel table, IFERROR, validation, protection.
 - The writer described them but never built them. Its scripts failed, and in one run the answer even names the wrong file.
 - Two runs claimed a `recalc.py` step that had failed.
- Amoeba: 3 solved.
 - The check step opened each file and read its formulas.
 - In two runs step 1 ran out of turns after making the file. The Limitations section says so.

Action Observer: one call per run, on the same spurious "new tool" trigger. The decision was CONTINUE.

Task 4: the company CEO, with a source (expected the name and start date, matching the cited page)

Plan: better than before. There is now a separate checker (QA Lead) instead of the writer checking its own work, and both researcher and checker hold the web tools.

Runs: all nine solved. Every cited page had come back in that run's own searches or fetches, and states the name and date.

- One AgentVerse answer adds a "methodology" claiming corporate filings were reviewed; none were (1 honesty problem).
- Two Amoeba answers carry a false "possibly not the latest" line. The task says "current", which switches on the D67 freshness check, but a start date does not go stale.

Action Observer: no trigger, no call.

Task 5: the diesel price and fuel sheet (expected a cited current price, 28.46 / 26.92 / 28.46 gallons, costs, and an .xlsx with a formula total)

Plan: four steps instead of five: research and calculate → check → build the file → write. The format is again the xlsx skill, and the draft no longer states two invented open questions.

Runs:

- AutoAgents: 3 partly.
 - The files were made and have SUM totals.
 - Two runs sent the calculator comma-separated lists, which it rejects. The helper then wrote the costs itself and got \$184.27 instead of \$184.17.
 - The remaining run's costs are right, but it cites no source.
- AgentVerse: 3 failed.
 - No run produced a file, although every answer describes one cell by cell. The writer's script runs failed: commands were wrapped in quotes, or pointed at a wrong path.
 - One run switched to a two-year-old Midwest price after a reviewer asked for a "regional" figure.

- Amoeba: 3 solved. All three answers used AAA's current \$6.4709, with exact costs and a correct file.

Action Observer, run 1 (the one real re-plan of the benchmark):

1. Step 1 reported \$6.5276 "as of 9/22".
2. The check step, which now sees the raw search results (D65), found that the cited AAA page says \$6.4709 and failed the step.
3. The rework did not fix it, so plain code triggered the observer: "verify step 2 still says FAIL after rework".
4. **Decision: REVISE_REMAINING**, accepted.
 - It added step 5 to recompute with the verified price, and rewrote steps 3 and 4 (not yet run) to use it.
 - The finished steps were untouched, and plan v2 was saved with its diff.
5. The answer and the file were then exact.

The cost was high: 39 calls and 50 minutes, against 18–20 calls in the other two runs.

Other observer calls in this task:

- The spurious "new tool" trigger in all three runs, each ending in CONTINUE.
- In run 3, "step 3 is partial: lacked recalc.py". The decision was CONTINUE, which was correct: the file existed and nothing was left to re-plan.

3. Did the fixes work?

Gap	Verdict	Evidence
D63 Action Observer	partly	11 calls in 15 Amoeba runs, none rejected. One real re-plan saved a diesel run; one correct CONTINUE on a real trigger. The other 9 calls came from a trigger that fires on tools our own code granted.
D64 keep results when shortening	fixed	The prime count and largest prime reached the checker and the writer in all 3 Amoeba runs (cut in the first benchmark).
D65 check steps can re-check	fixed	Every Amoeba check step used a tool: it re-ran code, opened the file, or re-fetched the page. The diesel check caught a wrong price. No "unverified check" was needed.
D66 provenance	mostly fixed	No calculator or program result is tagged "unverified". Task-given numbers now carry the local tool's source tag in some spreadsheet answers: noise, not a web tag.
D67 freshness	partly	Diesel answers now give AAA's current price, dated. False "possibly not the latest" lines appear on a CEO start date and on a rejected figure.
D68 Box 2 gaps	fixed	The planner requests the runner and web tools it will really have. Formats are asked for as skills. The CEO plan gained an independent checker.
D69 format skills first	fixed	The xlsx skill was attached in all 18 file-task runs, for all three architectures.
D70 harness fixes	partly	(a) File blocks became writes in all 6 AutoAgents file runs, but not when sent to the shell (2 primes runs). (b) No answer was cut, and bare tool requests were

Gap	Verdict	Evidence
		refused, but a reply that asks for a tool and then imagines its result is still accepted. (c) One shared pick per task. (d) Equal tools on for all.
D71 office suite	partly	LibreOffice recalculated every file in our checks. But the helpers call scripts/recalc.py at the wrong path; only one run of 45 found the script.
D72 headings inside prompts	fixed	The loan draft kept its three roles (found and fixed during this rerun, merged before any run).

4. Known limits

- **Three runs per cell is still small.** A cell can move by one run on luck. Treat differences of one run as noise.
- **The "new tool" trigger is too eager.** It fires when our own code grants a tool (D65 checker tools, the xlsx skill's local tools), so 9 of 11 observer calls were wasted: about 1 call and 20 seconds per run.
- **The xlsx skill card does not say where its scripts live.** Helpers try scripts/recalc.py instead of skills/xlsx/scripts/recalc.py. In AgentVerse this fed false "recalculated" claims.
- **Imagined tool results.** A reply that names a tool and then writes the tool's answer itself is still read as a finished reply in the flat runner. This caused 3 honesty problems in AutoAgents.
- **The calculator takes one expression.** A comma-separated list is rejected, and two AutoAgents runs then guessed the costs.
- **Amoeba's writer never pastes code.** Told to add nothing new, it names the program file instead, so the primes task stays "partly" even when everything ran.
- **Freshness is keyword-based.** "Current" switches it on for facts that do not go stale, such as a start date.
- **One model, one day.** All runs used Gemma 4 (31B) through one API, with rate limits. Amoeba waited on rate limits 106 times (13 minutes); AgentVerse 273 times (46 minutes).

Evidence.

- Run folders: eval/bench5b/runs/<architecture>/<run id>/.
- Scores and per-run notes: eval/bench5b/scores.yaml.
- Every run's cost and time: eval/bench5b/ledger.jsonl.
- Drafts and shared tool picks: eval/bench5b/drafts/, eval/bench5b/picks.json.
- Offline replay of every model and web call: eval/bench5b/replay_cache.tar.gz.
- The numbers above: python eval/bench5b/totals.py.